

# Imp Language Extension in Haskell

Formal Semantics & Verification

---

Calinov Cosmin

March 9, 2026

Introduction

Syntax & Semantics

Verification

Conclusions

# Introduction

---

# Project Goals

- implement an extended version of the IMP language
- support for:
  - local variables (blocks)
  - arithmetic operations (power, modulo)
  - parallel execution
- verify semantics using QuickCheck

# Syntax & Semantics

---

# Abstract Syntax

```
1 data Stmt
2   = Assign String Exp
3   | Seq Stmt Stmt
4   | If Exp Stmt Stmt
5   | While Exp Stmt
6   | Block Stmt      -- New scope
7   | Par Stmt Stmt   -- Parallelism
8   deriving (Show, Eq)
```

We implemented small-step semantics to handle concurrency:

$$\frac{\langle s_1, \sigma \rangle \rightarrow \langle s'_1, \sigma' \rangle}{\langle s_1 \parallel s_2, \sigma \rangle \rightarrow \langle s'_1 \parallel s_2, \sigma' \rangle}$$

This allows generic interleaving of instructions.

# Verification

---

# Testing with QuickCheck

**Challenge:** Random While loops often don't terminate.

**Solution:** Bounded Generation (Fuel).

```
1 -- Generator ensures loop strictly decreases a counter
2 safeWhile :: Int -> Gen Stmt
3 safeWhile fuel = do
4     -- ... generation logic ...
```

## Conclusions

---

- Extended IMP with Block, Par, arithmetic.
- Formal semantics guided the Haskell implementation.
- Property-based testing verified key invariants (determinism for non-parallel code).